



DEPARTMENT OF SOFTWARE ENGINEERING AND INFORMATION SYSTEM
FACULTY OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

CCS3600-6 ARTIFICIAL INTELLIGENCE
SEMESTER 2 2025/2026

TECHNICAL DEVELOPMENT MANUAL
ANTI-GROOMING DETECTION CHAT APPLICATION

THEME: INTELLIGENT ANTI-HARASSMENT SOLUTIONS
FOCUS AREA: IDENTIFYING GROOMING BEHAVIOUR

LECTURER: DR. NAZLEEN SYAHIRA BINTI MOHD SUHAIMI

No.	Name	Matric no.
1	Lim Ye Xuan	230847
2	Tye Zi Li	231373
3	Shawn Song-Yang Hee	227845
4	Chong Yu Ming	228714
5	Qistyn Nur Aneesa Mohd Zamri	229922
6	Zhou Mo	227218
7	Ding Zihan	226669

1.0 Introduction and Global Scenario.....	4
1.1 The Global Challenge: Online Grooming on University Campuses.....	4
1.2 Project Objective.....	4
1.3 Scope and Significance.....	5
2.0 System Architecture.....	6
2.1 Architectural Overview.....	6
2.2 Module Breakdown.....	6
2.2.1 Activities (Screens).....	6
2.2.2 Utilities.....	7
2.2.3 Models.....	7
2.2.4 Adapters.....	7
2.2.5 Firebase Integration.....	8
2.3 Tech Stack.....	8
3.0 Development Environment Setup.....	9
3.1 Prerequisites.....	9
3.3 Firebase Configuration.....	9
3.3.1 google-services.json.....	9
3.3.2 Firebase Admin SDK Service Account.....	10
4.0 Code Logic and AI Algorithms.....	11
4.1 Core Detection Engine: GroomingDetector.java.....	11
4.1.1 Algorithm Description.....	11
4.1.2 Keyword Weight Design.....	11
4.1.3 Code Walkthrough.....	12
4.2 ChatActivity: Detection Integration.....	13
4.3 RiskDashboardActivity: Conversation-Level Analysis.....	14
4.4 SafetyHubActivity: Gemini AI Chatbot.....	14
4.4.1 System Prompt Engineering.....	14
4.4.2 Multi-turn Conversation Management.....	15
4.4.3 Threading.....	15
4.5 AccessTokenManager: OAuth2 Token Management.....	15
5.0 Critical Review of Algorithm Suitability.....	16
5.1 Strengths of the Rule-Based Keyword Scoring Approach.....	16
5.1.1 Interpretability and Transparency.....	16
5.1.2 Low Latency.....	16

5.1.3 Privacy Preservation.....	16
5.1.4 No Training Data Required.....	16
5.2 Limitations and Critical Weaknesses.....	17
5.2.1 High False Positive Rate.....	17
5.2.2 Trivial Evasion by Motivated Attackers.....	17
5.2.3 Unidirectional Analysis.....	17
5.2.4 Static Keyword Dictionary.....	18
5.2.5 Absence of Natural Language Processing.....	18
5.3 Suitability Assessment for University Setting.....	18
 6.0 Security and Privacy Considerations.....	 20
6.1 API Key Management.....	20
6.2 Service Account Credentials.....	20
6.3 Firestore Data Retention.....	20
6.4 Survivor Data Sensitivity.....	20
 7.0 Conclusion.....	 21

1.0 Introduction and Global Scenario

1.1 The Global Challenge: Online Grooming on University Campuses

Online grooming is the process by which a predator systematically builds trust with a potential victim through digital communication channels to facilitate sexual abuse or exploitation, which represents one of the most pressing and underreported forms of sexual harassment in the modern era. While commonly associated with the exploitation of minors, grooming behaviour is equally prevalent and dangerous in university and young adult environments, where students are navigating new social freedoms, digital relationships, and unfamiliar peer dynamics.

According to global child safety organisations, the majority of grooming incidents begin on publicly accessible digital platforms such as social media, messaging apps, and online gaming environments. On university campuses specifically, students are particularly vulnerable due to:

- Social isolation, especially during the first year of enrollment
- Reduced parental oversight and increased digital autonomy
- High reliance on mobile chat applications for social interaction
- Limited awareness of what constitutes predatory communication patterns
- Cultural or social stigma that discourages reporting

Critically, most existing chat platforms provide no mechanism whatsoever for identifying or flagging grooming language in real time. Survivors often only recognise the danger retrospectively, when harm has already occurred. This gap between the prevalence of grooming behaviour and the absence of proactive detection tools defines the core problem that this application addresses.

1.2 Project Objective

This project develops an Android-based chat application that integrates AI-powered grooming behaviour detection directly within the messaging interface. The application serves a dual function:

1. Real-time grooming detection: Analysing outgoing messages for language patterns associated with grooming behaviour before they are sent, using a rule-based keyword scoring engine.

2. AI-powered survivor support: Providing a dedicated Safety Hub chatbot, powered by Google Gemini, that offers survivors compassionate, context-aware guidance, reporting pathways, and emotional support.

The application is mobile-first, requires no specialised hardware, and is designed to function within the everyday digital communication habits of university students, thereby removing barriers to awareness and support.

1.3 Scope and Significance

The significance of this application extends beyond individual protection. By logging flagged incidents to a secure cloud backend (Firebase Firestore), the system creates an auditable record of grooming-related communications that can support institutional reporting, counselling referrals, and policy development. The Risk Dashboard feature allows users to visualise the cumulative risk profile of a conversation over time, transforming individual message-level detection into a holistic conversation-level safety assessment.

2.0 System Architecture

2.1 Architectural Overview

The application follows a layered Android architecture pattern without an explicit MVVM or MVP framework, the logic is distributed across Activity classes with a clear separation between UI rendering, business logic, and data persistence. The architecture is summarised in the table below:

Layer	Technology	Responsibility
Presentation	Android XML Layouts	UI rendering, user interaction, data binding
Business Logic	Java (Activities, Adapters)	Grooming detection, chat flow, navigation
AI / Detection	GroomingDetector.java, Gemini API	Keyword scoring, risk classification, LLM chat
Data	Firebase Firestore	Messages, users, flagged incidents, conversations
Messaging	Firebase Cloud Messaging (FCM)	Push notifications via Retrofit + OAuth2 token
Networking	Retrofit 2 + OkHttp	FCM API calls, token management

2.2 Module Breakdown

2.2.1 Activities (Screens)

Each screen is implemented as an Android Activity, using View Binding for type-safe access to UI components:

Activity	Purpose
SignInActivity	User authentication via Firebase Auth email/password
SignUpActivity	New user registration with profile image capture and Base64 encoding
MainActivity	Home screen: lists recent conversations and online users; manages FCM token
UsersActivity	Browse registered users to initiate a new conversation

ChatActivity	Core chat UI with real-time grooming detection on outgoing messages
RiskDashboardActivity	Analyses all messages in a conversation to compute a cumulative risk score
SafetyHubActivity	AI chatbot (Gemini) providing survivor support, guidance, and resources
ReportActivity	Structured incident reporting form pre-populated with flagged message context
BaseActivity	Abstract base class managing online/offline availability status in Firestore

2.2.2 Utilities

- GroomingDetector.java: Core keyword scoring and classification engine (see Section 4 for detailed analysis)
- PreferenceManager.java: Wrapper around SharedPreferences for persistent user session data
- AccessTokenManager.java: Handles Google OAuth2 token refresh for FCM HTTP v1 API authentication
- Constants.java: Centralised string constants for Firestore field names, intent extras, and API keys
- MarkdownUtils.java: Strips markdown formatting from Gemini API responses for clean display

2.2.3 Models

- ChatMessage.java: Data model for individual chat messages (senderId, message, dateTime, isFlagged, riskScore, riskLevel)
- User.java: Data model for user profiles (id, name, image, token)
- SafetyHubMessage.java: Data model for Safety Hub AI conversation messages

2.2.4 Adapters

- ChatAdapter.java: RecyclerView adapter rendering sent and received chat bubbles; colour-codes flagged messages

- RecentConversationsAdapter.java: Renders the conversation list on the home screen
- UsersAdapter.java: Renders the user browse list
- SafetyHubAdapter.java: Renders the Safety Hub chat interface

2.2.5 Firebase Integration

- MessagingService.java: Extends FirebaseMessagingService; handles incoming push notifications and FCM token refresh
- Firestore Collections: users, chat, conversations, flagged_incidents

2.3 Tech Stack

Layer	Technology
Language	Java (Android SDK)
UI	XML layouts, View Binding, RecyclerView
Database	Firebase Firestore
Push Notifications	Firebase Cloud Messaging (FCM)
AI / LLM	Google Gemini API (gemini-flash-latest)
HTTP Client	Retrofit 2
Markdown Rendering	Markwon

3.0 Development Environment Setup

3.1 Prerequisites

Ensure the following tools are installed on the development machine before cloning the project:

Tool	Version	Notes
Android Studio	Hedgehog (2023.1.1) or later	Recommended: Meerkat 2024.3.1+
JDK	Java 17 (LTS)	Bundled with Android Studio
Android SDK	API 34 (Target), API 24 (Min)	Install via SDK Manager
Gradle	9.4.1	Managed by Gradle wrapper
Google Account	Any	Required for Firebase Console access
Git	2.x	Records snapshots of code and configuration files

3.3 Firebase Configuration

The application requires a Firebase project with the following services enabled:

- Firebase Authentication (Email/Password provider)
- Cloud Firestore (Native mode, multi-region or regional)
- Firebase Cloud Messaging (FCM)
- Firebase Storage (optional, for profile images if extended)

3.3.1 google-services.json

The `google-services.json` file is generated in the Firebase Console under Project Settings. It contains the project ID, API keys, and OAuth client configurations required by the Firebase SDK at runtime.

3.3.2 Firebase Admin SDK Service Account

The file `chatapp-334b8-firebase-adminsdk-fbsvc-029f2931af.json` is the Firebase Admin SDK service account credentials file. It is used by `AccessTokenManager.java` to generate short-lived OAuth2 bearer tokens for the FCM HTTP v1 API (which replaced the legacy FCM API in 2024).

4.0 Code Logic and AI Algorithms

4.1 Core Detection Engine: GroomingDetector.java

4.1.1 Algorithm Description

The `GroomingDetector` class implements a weighted keyword scoring algorithm structured around two stages: feature extraction (keyword matching) followed by threshold-based classification.

Stage 1

Feature Extraction: The input message string is normalised to lowercase. Each keyword in the `GROOMING_KEYWORDS` `HashMap` is tested for substring presence using `String.contains()`. When a match is found, its corresponding weight (risk score) is added to a running total, and the matched keyword is recorded for the reason string.

Stage 2

Classification (Threshold-Based Branching): The accumulated score is evaluated against two predefined thresholds to determine the appropriate system response:

Score Range	Risk Level	Action Triggered
≥ 50	HIGH	Message blocked; incident logged to Firestore; user redirected to <code>ReportActivity</code>
20 – 49	MEDIUM	<code>AlertDialog</code> warns user; user may choose to send anyway (message flagged in Firestore)
0 – 19	SAFE	Message sent normally with no intervention

4.1.2 Keyword Weight Design

The keyword dictionary is designed on a severity gradient: phrases indicating explicit requests for secrecy, sexual content, or physical isolation carry the highest weights (30–50 points), while softer indicators of manipulation or trust-building carry lower weights (5–20 points). This ensures that a

single high-severity keyword can immediately trigger a HIGH alert, while a pattern of lower-severity terms accumulates to a MEDIUM warning:

Keyword / Phrase	Score	Rationale
nude	50	Direct solicitation of explicit imagery — immediate HIGH threshold breach
private photo	40	Request for private or intimate imagery
meet alone	40	Isolation tactic — key grooming escalation behaviour
parents won't know	35	Secrecy enforcement targeting parental oversight
our secret	30	Secrecy normalisation — core grooming tactic
don't tell	30	Instruction to conceal communication
sexy	25	Sexualisation of conversation
webcam	20	Request for live video — potential voyeurism indicator
lonely	15	Emotional manipulation — exploiting vulnerability
gift / money	10	Transactional grooming — offering inducements
sweetie / darling	10	Inappropriate intimacy — boundary violation language

4.1.3 Code Walkthrough

The full `analyze()` method logic:

```
public static DetectionResult analyze(String message) {  
    String lowerMessage = message.toLowerCase();  
    int score = 0;  
    StringBuilder detectedPatterns = new StringBuilder();  
  
    // Stage 1: Feature Extraction  
    for (Map.Entry<String, Integer> entry :  
        GROOMING_KEYWORDS.entrySet()) {  
        if (lowerMessage.contains(entry.getKey())) {
```

```

        score += entry.getValue();
        detectedPatterns.append(entry.getKey()).append(",
");
    }
}

// Stage 2: Classification (Threshold-Based Branching)
if (score >= 50) {
    return new DetectionResult(RiskLevel.HIGH, score, "High
risk: " + detectedPatterns);
} else if (score >= 20) {
    return new DetectionResult(RiskLevel.MEDIUM, score,
"Suspicious: " + detectedPatterns);
} else {
    return new DetectionResult(RiskLevel.SAFE, score, "No
significant risk");
}
}

```

4.2 ChatActivity: Detection Integration

In `ChatActivity.sendMessage()`, the keyword scoring result is evaluated before any Firestore write occurs. This pre-send interception ensures flagged messages are caught at the point of composition such as blocking or warning the sender before the message is delivered:

```

private void sendMessage() {
    String messageText =
binding.inputMessage.getText().toString();

    GroomingDetector.DetectionResult result =
GroomingDetector.analyze(messageText);

    if (result.riskLevel == RiskLevel.HIGH) {
        showHighRiskAlert(result); // Block and log
    } else if (result.riskLevel == RiskLevel.MEDIUM) {

```

```

        showMediumRiskWarning(result); // Warn and allow
override
        } else {
            performSendMessage(messageText, false, 0, null, null);
        }
    }
}

```

When a HIGH-risk message is blocked, `logHighRiskIncident()` writes the sender ID, receiver ID, full message text, timestamp, risk score, and risk level to the `flagged_incidents` Firestore collection for administrative review.

4.3 RiskDashboardActivity: Conversation-Level Analysis

The Risk Dashboard extends message-level detection to a conversation-level risk assessment. It queries all messages sent from the receiver to the current user, applies `GroomingDetector.analyze()` to each, and aggregates the results:

- `totalScore`: sum of risk scores across all non-SAFE messages
- `flaggedCount`: count of messages that triggered MEDIUM or HIGH classification
- `detectedWords`: consolidated list of all matched keywords across the conversation

The cumulative score is normalised to a 0–100 scale (capped at 100) and displayed as a progress bar with colour-coded risk labels. This gives users a visual, intuitive understanding of the overall danger profile of a conversation over time, transforming point-in-time detection into a longitudinal safety signal.

4.4 SafetyHubActivity: Gemini AI Chatbot

The Safety Hub provides a conversational AI interface powered by Google Gemini (gemini-2.0-flash model). Its implementation involves several noteworthy design choices:

4.4.1 System Prompt Engineering

Before the user's first message, `SafetyHubActivity` constructs a system-level prompt that:

- Instructs the model to act as a compassionate, trained counsellor for sexual harassment survivors.
- Provides the full text of the flagged conversation from another party as context, allowing the AI to give specific, relevant advice rather than generic responses.
- Instructs the model to provide reporting pathways and emergency resources when appropriate.
- Instructs the model to avoid victim-blaming language.

4.4.2 Multi-turn Conversation Management

The `chatHistory` `JSONArray` maintains the full conversation history in the format required by the Gemini API (`{role: 'user'/'model', parts: [{text: '...'}]}`). Each new message appends to this array, and the full history is sent with every API call, enabling the model to maintain context across multiple turns of the conversation — essential for coherent, personalised support interactions.

4.4.3 Threading

The Gemini API call runs on a background thread via a single-threaded `Executor` (`Executors.newSingleThreadExecutor()`), preventing UI thread blocking. The UI is updated by posting a `Runnable` to the main thread handler after the response is received, following Android's threading model for network operations.

4.5 AccessTokenManager: OAuth2 Token Management

Firebase FCM HTTP v1 API requires a short-lived OAuth2 bearer token generated from the service account credentials. `AccessTokenManager.java` uses the Google Auth Library to:

1. Load the service account JSON from the app's assets directory.
2. Create a `GoogleCredentials` object scoped to `firebase.messaging`.
3. Automatically refresh the token when it is within 5 minutes of expiry.
4. Cache the token in memory and provide it asynchronously via a callback interface.

This eliminates the need for the deprecated FCM legacy server key and complies with Google's FCM HTTP v1 requirements.

5.0 Critical Review of Algorithm Suitability

5.1 Strengths of the Rule-Based Keyword Scoring Approach

5.1.1 Interpretability and Transparency

The most significant advantage of the keyword-weighted scoring approach is its complete interpretability. Unlike deep learning models (neural networks, transformers), the detection logic is fully readable and auditable by non-technical administrators, university counsellors, and legal teams. Every triggered keyword is surfaced in the reason string shown to the user, making the system's decisions explainable and contestable — a critical requirement in a sensitive welfare context.

5.1.2 Low Latency

The algorithm operates entirely on-device with $O(k)$ time complexity, where k is the number of keywords in the dictionary. On modern Android hardware, analysis of a typical chat message completes in under 1 millisecond. This imperceptible latency is essential for a real-time messaging application where any noticeable delay between typing and sending would degrade user experience.

5.1.3 Privacy Preservation

Because detection runs locally on the device (except for the Safety Hub, which sends data to Gemini), message content is never transmitted to a third-party server for analysis. This is a significant privacy advantage over cloud-based NLP services, and reduces the regulatory burden around sensitive data handling (e.g., PDPA in Malaysia, GDPR in Europe).

5.1.4 No Training Data Required

The rule-based approach does not require a labelled training dataset for model training. The keyword dictionary was manually curated by the research team, informed by grooming behaviour patterns identified in the PAN 2012 Sexual Predator Identification Dataset and established criminology literature. This approach allows domain experts (child safety professionals, university counsellors) to assemble and validate the lexicon without machine learning expertise, enabling deployment without the significant data engineering overhead of a trained model. The ethical concerns around curating sensitive grooming datasets for training purposes are also avoided entirely.

5.2 Limitations and Critical Weaknesses

5.2.1 High False Positive Rate

The substring matching approach has no understanding of context, sentiment polarity, or conversational intent. The keyword 'don't tell' will trigger a detection even in a casual, supportive conversation ('Don't tell your mom about her birthday surprise yet!'). Similarly, 'gift' will flag any message about birthday presents. In a university setting where students communicate about everyday social activities, these false positives risk desensitising users to warnings, leading to alert fatigue where genuine risk signals are ignored.

5.2.2 Trivial Evasion by Motivated Attackers

A predator who is aware of keyword-based filtering can trivially evade detection by:

- (a) deliberate misspelling ('n_ude', 'se*xy');
- (b) character substitution using lookalike Unicode characters;
- (c) splitting keywords across messages; or
- (d) using synonyms, slang, or coded language specific to their target community(e.g., 'Can I rizz you?' is a contemporary slang that conveys predatory intent but contains no flagged keywords)

This is a well-documented limitation of all keyword-based content moderation systems.

5.2.3 Unidirectional Analysis

The current implementation only analyses outgoing messages sent by the current user. In a real grooming scenario, the predator is the sender, but the grooming messages arrive as incoming messages from the receiver, which are not currently analysed by GroomingDetector. The Risk Dashboard partially addresses this by analysing all messages from the receiver in the Firestore query, but the real-time interception only applies to the current user's outgoing messages. Additionally, the conversation context passed to the Safe Hub's Gemini API contains only messages received from the other party, not the current user's replies. This limits the AI counsellor's ability to assess the full conversational dynamic when providing guidance.

Potential fix: Extend GroomingDetector.analyze() to run on incoming messages as they are received in the listenMessages() callback, displaying a passive risk indicator rather than blocking (since the user is the recipient, not the sender). The Gemini context limitation could be addressed by querying both sent and received messages when building the conversation string in RiskDashboardActivity.

5.2.4 Static Keyword Dictionary

The keyword set is compiled at development time and cannot be updated without releasing a new APK. Grooming language evolves continuously, incorporating new platform-specific slang, emoji-based communication, and culturally specific terms. A static dictionary becomes outdated rapidly in production.

Potential fix: Load the keyword dictionary from a remote Firebase Remote Config or Firestore document, enabling hot-updates without requiring an app release.

5.2.5 Absence of Natural Language Processing

The algorithm performs no tokenisation, lemmatisation, part-of-speech tagging, or semantic analysis. It treats the message as a raw character sequence, missing contextual signals that NLP-based approaches would capture. For example, it cannot distinguish between 'I don't tell anyone when I'm upset' (SAFE) and 'Don't tell anyone about this' (HIGH) because both contain 'don't tell'.

This limitation is a deliberate consequence of the deployment constraints. NLP-based classification requires either a pre-trained model running on-device via a framework such as TensorFlow Lite, or a cloud-based inference server processing messages in real time. The former requires significant model optimization and increases APK size substantially; the latter introduces latency, privacy concerns, and infrastructure costs that are beyond the scope of a mobile-first prototype without a dedicated backend server. The rule-based approach was therefore selected as the most practical solution within these constraints, with the Gemini-powered Safe Hub providing contextual language understanding for user-initiated queries where latency is acceptable.

5.3 Suitability Assessment for University Setting

For deployment in a university campus environment, the current algorithm provides an adequate first-generation protective layer, particularly for survivors who may not recognise grooming behaviour in real time. The combination of immediate message-level alerts, conversational risk visualisation, and AI-powered survivor support creates a meaningful safety ecosystem that did not previously exist in campus chat applications.

However, for a production deployment serving a large student population, the algorithm's limitations must be addressed through iterative improvement. The potential development roadmap is:

Phase	Enhancement	Expected Impact
1	Analyse incoming messages passively	Detect grooming from the sender's side in real time
2	Remote keyword dictionary	Enable rapid response to emerging grooming language patterns
3	NLP-based context analysis	Reduce false positives; detect negation, sarcasm, and context
4	Machine learning classifier (trained on labelled data)	Adapt to evasion tactics; detect novel grooming patterns
5	Multilingual keyword expansion	Expand the keyword dictionary and localization to support non-English grooming language patterns

6.0 Security and Privacy Considerations

6.1 API Key Management

The current implementation stores the Gemini API key as a plaintext constant in Constants.java. While acceptable for a prototype, this practice exposes the key to extraction via APK decompilation. For production deployment, the key should be proxied through a backend server (e.g., Firebase Cloud Functions) so it is never embedded in client-side code.

6.2 Service Account Credentials

The Firebase Admin SDK service account JSON file, stored in the app's assets, contains an RSA private key. This must be treated as a secret and excluded from public version control. In production, FCM notifications should be dispatched from a server-side Cloud Function rather than from the Android client, eliminating the need to distribute service account credentials at all.

6.3 Firestore Data Retention

Flagged incidents are stored indefinitely in Firestore under the flagged_incidents collection. A data retention policy should be implemented (e.g., auto-deletion after 12 months) to comply with privacy regulations. Access to this collection should be restricted to administrative service accounts only.

6.4 Survivor Data Sensitivity

The application handles highly sensitive personal data related to sexual harassment. All Firestore communications are encrypted in transit via TLS. Firestore at-rest encryption is provided by Google by default. The application does not collect biometric data or device identifiers beyond the FCM token.

7.0 Conclusion

The Anti-Grooming Detection Chat Application represents a practical, deployable response to the underaddressed problem of online grooming on university campuses. By embedding AI-powered detection directly within the messaging interface, the precise point of risk, the application breaks from the traditional reactive model of harassment intervention (report after the fact) and introduces proactive, real-time protective intelligence.

The keyword-weighted rule-based keyword scoring approach provides a transparent, low-latency, and privacy-preserving first line of defence that is appropriate for a first-generation deployment. Its limitations, susceptibility to evasion, false positives, and static vocabulary are well-understood and addressable through the phased enhancement roadmap described in Section 5.3.

The integration of Google Gemini as a Safety Hub counsellor extends the application's value beyond detection into survivor support, providing context-aware, compassionate guidance that complements the analytical capabilities of the detection engine. Together, these features create a holistic safety ecosystem that addresses both the identification of grooming behaviour and the needs of those affected by it.

This application demonstrates that meaningful AI-powered anti-harassment tools can be built with standard mobile development frameworks, publicly available AI APIs, and cloud infrastructure without requiring specialised hardware or significant institutional investment, which makes it viable for rapid deployment across university campuses at scale.